



Programación de Estructuras de Datos y Algoritmos Fundamentales

Actividad Integradora 1 Estructuras de datos jerárquicas

Farid Gabriel Velasco Martínez A01736669

14 de mayo de 2025

Índice

Índice	2
Introducción	3
Análisis del algoritmo de ordenamiento	4
Análisis del algoritmo de búsqueda 1	5
Análisis del algoritmo de búsqueda 2	6
Conclusión	7

Introducción

En esta tercera actividad integradora tomamos un enfoque completamente diferente, dada la información existente en los registros podemos extraer fechas, mensajes de error, pero crucialmente la IP que es la ‘ubicación’ digital única de cada dispositivo conectado a la red, esta información nos permite rastrear desde donde se originó el intento de ataque, o bien marcar esa IP como sospechosa y estar preparados cuando se intente acceder a nuestra red desde ella de nuevo. Las estructuras de datos lineales son versátiles y aún son útiles en este caso, sin embargo tienen ciertas limitaciones, sus métodos de ordenamiento, búsqueda y operaciones específicas no siempre son los más eficientes, además de que a una mayor cantidad de datos pueden volverse ineficientes o difíciles de manejar, para sobrepasar estas limitaciones podemos hacer uso de otro tipo de estructuras de datos.

Para los objetivos de esta actividad integradora implementamos un heap que es una estructura de datos jerárquica que nos permite almacenar información y analizarla eficientemente tanto temporal como espacialmente. Este tipo de estructuras más avanzadas se acercan más a las usadas en el ámbito de producción ya que son más robustas.

Beneficios del heap

Un heap consiste en un árbol binario completo, es decir todos sus nodos padre están completos a excepción del último nodo y además sigue la premisa de que cada padre es mayor que sus hijos si es Max Heap o que cada padre es menor que sus hijos si es Min Heap y se ordena siempre de izquierda a derecha, la cual es la propiedad que hace que se mantenga como un árbol binario completo.

En un BST normal, es muy común que estos estén desbalanceados, es decir, que uno de los lados del árbol tenga más elementos que el otro, que no todos los nodos estén completos, además de que es imperativo que se usen nodos especiales, es decir clases con punteros a otros nodos, lo cual lo hace bastante más complejo de manejar y degrada la eficiencia de sus métodos, por ejemplo el método `push()` en un BST se degrada a $O(n)$ mientras que en un Heap siempre es $O(\log n)$. En un Heap es posible guardar la información en un Heap ya que el acceso a cada uno de sus nodos puede hacerse mediante las siguientes expresiones matemáticas:

$$\text{Nodo Padre} = (i - 1) \div 2$$

$$\text{Nodo Hijo Izquierdo} = i \times 2 + 1$$

$$\text{Nodo Hijo Derecho} = i \times 2 + 2$$

Siendo i el índice desde el cual buscar

Se podría argumentar que el uso de vectores como forma de almacenamiento de un heap es ineficiente ya que evita la capacidad de hacer crecer dinámicamente la estructura, sin embargo eso es equivocado, al integrar métodos como `set_capacity()` que reservan espacio del vector aseguramos que no se realicen relocalaciones de memoria constantemente, además de que acceder a un nodo específico tiene complejidad de $O(1)$ mientras que en un BST en el peor caso se degrada de nuevo a $O(n)$ y por último crucialmente el acceso al nodo mayor o menor en un BST tiene una complejidad de $O(\log n)$ o $O(n)$ en el peor caso, mientras que dependiendo de si el heap es max o min tendrá ya sea $O(1)$ o $O(n/2) = O(n)$. Por tanto no solo es más eficiente temporalmente si no también espacialmente usar un Heap.

Análisis de la complejidad computacional de las operaciones básicas del heap

Las operaciones básicas en el heap implementado en esta actividad son las siguientes:

Push: $O(\log n)$

Utiliza `heapify_up()` para reparar el heap, esta operación tiene una complejidad baja y es eficiente

Pop: $O(\log n)$

Utiliza `heapify_down()` para reparar el heap hacia abajo y tiene la misma complejidad que el anterior

`peak_max()` / `peek_min()`:

Dependiendo de la configuración del heap si es Max, `peek_max()` tiene una complejidad de $O(1)$, mientras que `peek_min()` aunque sólo recorre la mitad del heap se reporta como una complejidad de $O(n)$, si el heap es Min estas complejidades se intercambian

`heap_sort()`: $O(n \log n)$

Este es uno de los algoritmos de ordenamiento más eficientes y es especialmente adecuado para un heap

La implementación de cada operación es eficiente, la complejidad mayor de todos sus métodos es $O(n \log n)$ y en términos de análisis de datos esto es muy eficiente, además debido

al diseño de nuestra clase, una vez que se llama al metodo `heap_sort()` se optimizan los `peeks` para que mejoren su complejidad a $O(1)$. Esta eficiencia temporal y computacional es deseada para un análisis rápido en situaciones como la presentada en la situación problema.

Pseudocódigo del algoritmo de búsqueda del mínimo elemento con condición

```
function find_min_if(fun):
```

```
    if heap está vacío:
```

```
        lanzar excepción "Heap vacío"
```

```
    min_ptr ← null
```

```
    // Buscar entre las hojas (de size/2 a size-1)
```

```
    para i desde (size / 2) hasta (size - 1) hacer:
```

```
        si fun(data[i]) es verdadero:
```

```
            si min_ptr es null o data[i] < *min_ptr:
```

```
                min_ptr ← referencia a data[i]
```

```
    // Si no se encontró ningún valor entre las hojas
```

```
    si min_ptr es null:
```

```
        // Buscar entre los padres (de 0 a size/2 - 1)
```

```
        para i desde 0 hasta (size / 2 - 1) hacer:
```

```
            si fun(data[i]) es verdadero:
```

```
                si min_ptr es null o data[i] < *min_ptr:
```

```
                    min_ptr ← referencia a data[i]
```

```
    si min_ptr sigue siendo null:
```

```
        lanzar excepción "Ningún elemento cumple la condición"
```

```
    devolver *min_ptr
```

Este enfoque garantiza una complejidad máxima de $O(n)$, si la condición está bien estructurada y es probable que el elemento que cumpla con dicha condición se encuentre en la segunda mitad del vector (nodos hoja) el algoritmo recorrerá como máximo el heap $n/2$ veces. Se utiliza la misma estrategia que para los métodos peek que dado un set de elementos desordenados, es la mejor alternativa.

Posteriormente este algoritmo se podría mejorar para optimizarse dado que el heap este ordenado o no.

Conclusión

Al utilizar una estructura de datos más avanzada como lo es un heap, obtenemos ventajas significativas en cuanto al análisis, recopilación y almacenamiento de los datos. Es crucial no solo optimizar las operaciones de cada estructura de datos si no conocer sus características para usar la más adecuada en cada situación y objetivo que se tenga. Por otro lado, no hace falta remarcar que entre más eficiente y computacionalmente sencillo sea un algoritmo, más capacidad de realizar análisis profundos de los datos tenemos. Al combinar estructuras de datos lineales con sus respectivos algoritmos eficientes y estructuras de datos jerárquicas, podemos encontrar patrones y generar conclusiones más acertadas sobre lo que representa un conjunto de datos.

Referencias:

1. *Acelera tus operaciones con IOT*. (s. f).
<https://www.oracle.com/mx/internet-of-things/>
2. Condliffe, J. (2020, 2 abril). The FBI Shut Down a Huge Botnet, but There Are Plenty More Left. *MIT Technology Review*.
<https://www.technologyreview.com/2017/04/11/152645/the-fbi-shut-down-a-huge-bot-net-but-there-are-plenty-more-left/>

3. GeeksforGeeks. (2022, 1 septiembre). *Minimum element in a max heap*.
GeeksforGeeks. <https://www.geeksforgeeks.org/minimum-element-in-a-max-heap/>
4. GeeksforGeeks. (2023, 10 octubre). *Insertion and Deletion in Heaps*. GeeksforGeeks.
<https://www.geeksforgeeks.org/insertion-and-deletion-in-heaps/>
5. GeeksforGeeks. (2025a, enero 2). *Heap Sort Data Structures and Algorithms Tutorials*. GeeksforGeeks. <https://www.geeksforgeeks.org/heap-sort/>
6. GeeksforGeeks. (2025b, marzo 19). *Priority Queue in C++ STL*. GeeksforGeeks.
<https://www.geeksforgeeks.org/priority-queue-in-cpp-stl/>
7. GeeksforGeeks. (2025c, marzo 21). *Building Heap from Array*. GeeksforGeeks.
<https://www.geeksforgeeks.org/building-heap-from-array/>
8. Inside code. (2021, 28 agosto). *Heaps, heapsort, and priority queues - Inside code*
[Video]. YouTube. <https://www.youtube.com/watch?v=pLIajuc31qk>
9. Spanning Tree. (2020, 16 julio). *What Is a Binary Heap?* [Video]. YouTube.
<https://www.youtube.com/watch?v=AE5I0xACpZs>
10. Robinson, S., Hanna, K. T., & Lutkevich, B. (2025, 18 febrero). *What is a botnet?*
Search Security.
https://www.techtarget.com/searchsecurity/definition/botnet?track=NL-1823&ad=931942&src=931942&asrc=EM_NLN_122529935